Developer Guide

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Pages . . . 2
Requirements . . . 2
Configure . . . 2
Compile . . . 3
Compiler options . . . 3
Linking . . . 3
Install . . . 3
Required software . . . 4
Starting Visual Studio . . . 4
Cloning the repository . . . 4
Configuring, compiling, running . . . 5
Resources . . . 6
Cost . . . 6
Starting an instance . . . 6
Building . . . 7
Running and debugging . . . 7
Using your own dataset . . . 7
Stopping the instance . . . 8
Deleting the instance . . . 8
Test script . . . 9
Running the test at 62.0.1 . . . 9
Adding a scenario . . . 9
Continuous integration . . . 10
Version numbers . . . 10
Version in the executable and outputs . . . 10
Building a tagged version . . . 10
Making a release . . . 11
Interoperability and portability . . . 11
Readability . . . 12
Reliability . . . 12
Arrays . . . 13
Dynamic memory and pointers . . . 13
Looping . . . 13
Functions and procedures . . . 14
I/O . . . 14
Obsolescent features to avoid . . . 14
Source files . . . 15
References . . . 15

Code Of Conduct **16**
Repositories . . . 16
Reporting a problem . . . 16
Contributing code . . . 16
Contributing to the documentation . . . 17

This section is for people working on the SWAT+ Fortran source: building from a clone, running the test suite, and submitting changes back to `swat-model/swatplus`.

If you only need to run a prebuilt SWAT+ executable, see Install instead.

Pages

- Build with CMake. The build used on every platform: configuration, compilers (gfortran, `ifx`, `ifort`), compiler options, debug and release builds.
- Visual Studio on Windows. Visual Studio 2022 with the Intel oneAPI Fortran compiler: cloning, configuring, and debugging against an input dataset.
- VS Code Codespaces. Running a SWAT+ build and debug session inside a GitHub Codespaces virtual instance. Useful when you do not want to install a Fortran toolchain locally.
- Testing. The scenario regression test that compares model output with reference outputs, and how to run it at 62.0.1.
- Tagging and releases. How the version number comes from the Git tag and reaches the executable, and how a release is made.
- Coding conventions. Fortran 90/95 style rules for SWAT+ source contributions: portability, readability, reliability, arrays, I/O, and obsolescent features to avoid.
- Contributing. Reporting problems, pull requests, and contributing to this documentation.
- Code of conduct. The Contributor Covenant, applied to the SWAT+ project community.

SWAT+ is built out of source: all intermediate and final files go into a build folder, so the source tree stays clean and several builds (different compilers, debug and release) can exist side by side.

The build is defined in `CMakeLists.txt` at the root of the repository. It compiles every `.f90` file in `src/` into one executable.

Requirements

- CMake 3.22 or newer
- a Fortran compiler: gfortran, Intel `ifx`, or Intel `ifort` (`ifort` is no longer included in Intel oneAPI from the 2025 release on)
- Git, which the build uses to find the version number. Configuration stops if Git is not installed; in a copy of the source that is not a Git clone (a downloaded archive), the version becomes `unknown`.
- `make` or Ninja

Configure

From the `swatplus` folder:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
```

This writes the build system into `build/` without compiling anything. Give a build type: `Release` for an optimized executable, `Debug` for one with run-time checks and debugger symbols. Without a build type neither the optimization nor the debug flags are added.

To choose the compiler:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release -D CMAKE_Fortran_COMPILER=gfortran
cmake -B build -D CMAKE_BUILD_TYPE=Release -D CMAKE_Fortran_COMPILER=ifx
```

On Windows, configure from the Intel oneAPI command prompt to use the Intel compilers, or use Ninja with gfortran:

```
cmake -B build -D CMAKE_GENERATOR_TOOLSET=fortran=ifx
cmake -B build -G "Ninja" -D CMAKE_Fortran_COMPILER=gfortran -D CMAKE_BUILD_TYPE=Release
```

`CMakePresets.json` also defines debug and release presets for gfortran (Linux, Windows, macOS), `ifx` and `ifort`; list them with `cmake --list-presets`.

Compile

```
cmake --build build -j 8
```

The executable is written to `build/` as

`swatplus-<version>-<compiler>-<system>_<arch>[-Rel|-Dbg][.exe]`

for example `swatplus-62.0.1-gnu-lin_x86_64-Rel`. The compiler part is `gnu`, `ifx` or `ifo` (Intel Fortran Classic), and the version comes from the Git tag (see Tagging and releases).

To start again from scratch, delete the build folder (`rm -rf build`, or `rd /s /q build` in Windows `cmd`). To remove only the compiled files and keep the configuration:

```
cmake --build build --target clean
```

Compiler options

Compiler	Always	Release	Debug
gfortran (Linux, macOS)	free form, no line length limit, implicit none, local variables set to zero, strict rounding	`-O`	`-Wall -fcheck=all -ffpe-trap=invalid,zero,overflow,unde -fsignaling-nans`
gfortran (Windows)	as above, plus `-fcheck=all` and the floating-point traps	`-O`	`-Wall`
`ifx`	free form, `-fpe0`, traceback	`-O`	`-warn all -O0 -check bounds`
`ifort`	free form, `-fpe0`, traceback	`-O`	`-warn all`

From 62.0.1 the gfortran run-time checks and floating-point traps are used only in Debug builds on Linux and macOS. Earlier releases also applied them to Release builds, and those executables could stop with an illegal instruction error on macOS when a system math routine raised a harmless floating-point flag.

Linking

- Linux, gfortran: linked statically, except on Arch, Arch-based and Fedora systems, which do not ship the static `libgfortran`; there the executable needs the gfortran run-time libraries at run time (`ldd <executable>` lists them).
- macOS, gfortran: the gfortran, quadmath and gcc run-time libraries are linked statically, so the executable runs on Macs without Homebrew's gcc installed. A fully static macOS executable is not possible.
- Intel compilers and Windows gfortran: linked statically.

Install

`cmake --install build` copies the executable to the root of the source folder, whatever `--prefix` is given. To put it somewhere else, copy the file from `build/` yourself.

Sources

- CMakeLists.txt: version from `git describe --tags` (lines 20 to 34), compiler flags (`fdialect`, `fdebug`, `frelease`) and static linking (lines 36 to 100), executable name (`SWATPLUS_EXE`), `install(TARGETS ${SWATPLUS_EXE} DESTINATION RUNTIME DESTINATION ${PROJECT_SOURCE_DIR})`.
- CMakePresets.json and doc/Building.md.

Visual Studio 2022 on Windows can build, run and debug SWAT+ with the Intel Fortran compiler. The steps are: install the tools, clone the repository, configure the CMake build, and run a debug session against an input dataset.

The screenshots for these steps are in the `doc/` folder of the SWAT+ source repository.

Required software

Install in this order. The Intel compiler requires Visual Studio to be present first.

1. Visual Studio Community 2022. During install, in the **Workloads** panel, check **Desktop development with C++**. That workload pulls in the CMake integration that SWAT+ needs.
2. Intel Fortran Compiler for Windows. Use the standalone installer. The current Intel oneAPI release ships `ifx`. The older `ifort` was deprecated in 2024 and is gone from the 2025 release. SWAT+ produces the same results with either Intel compiler.
3. Git for Windows. Visual Studio includes a copy of Git, but SWAT+ uses the standalone `git` during the CMake build to read the current tag for the version string.

Reboot after the three installs finish.

Optional: install Python (from the Microsoft Store) and the Python workload in Visual Studio if you want to run the integrated test suite from inside the IDE.

Starting Visual Studio

The Intel compiler only works with the Visual Studio CMake integration if Visual Studio is launched from the Intel oneAPI command prompt. Follow these steps every time.

1. Open the Windows Start Menu, type `intel`, and select **Intel oneAPI command prompt for Intel 64 for Visual Studio 2022**. Pin it if you use it often.
2. The terminal opens and prints the oneAPI initialization output:

   ```
   :: initializing oneAPI environment...
      Initializing Visual Studio command-line environment...
      Visual Studio version 17.13.0 environment configured.
      "C:\Program Files\Microsoft Visual Studio\2022\Community\"
      Visual Studio command-line environment initialized for: 'x64'
   :  compiler -- latest
   :  debugger -- latest
   :: oneAPI environment initialized ::
   ```

3. At the prompt, run:

   ```
   devenv
   ```

   This launches Visual Studio with the Intel environment variables set.

If you start Visual Studio by clicking the desktop icon directly, the Intel compiler will not be found and the build will fail.

Cloning the repository

Visual Studio can clone the SWAT+ repository and open it as a CMake project in one step.

1. On the Visual Studio start screen, click **Clone a Repository**.
2. Click the GitHub icon (lower left) to browse your repositories. Sign in if prompted. Select your fork of `swatplus` and click **Open**. Adjust the local path if needed.
3. Visual Studio clones the repository and loads it into Solution Explorer as a CMake project. It runs the CMake configure step against whichever Fortran compiler it finds (`ifx`, `ifort`, or `gfortran`). The 2025 Intel compiler only ships `ifx`, so any `ifort` configurations will fail on that version.

Configuring, compiling, running

Configure

SWAT+ is a CMake project. The Visual Studio toolbar exposes the available compiler and build-type combinations through a drop-down. Switching the selection triggers a reconfigure. The Output window prints the CMake settings used; check there which compiler was picked up.

To force a fresh configure, right-click `CMakeLists.txt` in Solution Explorer and select **Delete Cache and Reconfigure**.

Build

Use **Build > Build All** or **Build > Rebuild All**. The executable name encodes the compiler, architecture, and version. The version is read from the latest Git tag automatically. See Tagging and releases.

The Intel-built Windows executable is dynamically linked. To run it on a machine without Visual Studio, ship the Intel redistributable libraries alongside it.

Run and debug

You need to tell the project which folder contains the input dataset to run against.

1. Right-click `CMakeLists.txt` in Solution Explorer and select **Add Debug Configuration**. If Co-Pilot is hiding Solution Explorer, close that pane first.
2. In the dialog, pick **Default** and click **Select**. Visual Studio creates `launch.vs.json` if it does not exist and opens it for editing.

The default file looks like this:

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "CMakeLists.txt"
    }
  ]
}
```

Add a `currentDir` entry pointing at your input dataset. The path can be absolute or relative to the `swatplus` project root. The `name` field is what shows up in the toolbar.

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "Ames_testing",
      "currentDir": "C:/work/sp/Ames_testing"
    }
  ]
}
```

Do not change `project` or `type`. You can add more configurations by repeating **Add Debug Configuration** and editing the new block.

Pick your configuration from the toolbar drop-down and run **Debug > Start Debugging** or **Debug > Start Without Debugging**. Breakpoints, conditional watches, and step-through all work.

Resources

- Intel C++ and Fortran Compilers Redistributable Libraries by Version
- Intel Fortran Compiler
- Intel Fortran Compiler for oneAPI release notes
- Visual Studio documentation
- CMake

  Sources

  - `doc/VS-Win.md` and its screenshots in the source repository, with `CMakeSettings.json` for the Visual Studio configurations.

The SWAT+ repository ships a `.devcontainer/` folder that lets GitHub Codespaces start a Visual Studio Code instance in the browser with the build tools installed, so no local Fortran compiler is needed.

When a Codespaces instance starts, it:

- loads your fork into a remote working folder
- installs the Intel `ifx` and `gfortran` compilers
- configures VS Code to use the CMake build system
- installs the VS Code extensions needed to debug Fortran, and the `fortls` Fortran language server
- creates `/workdata/my_data` and copies the `refdata/Ames_sub1` dataset into it

In 62.0.1 the `/workdata/Ames_sub1` folder that the default `Ames_sub1` debug configuration points to is not created. To use that configuration, create the folder and copy `refdata/Ames_sub1` into it from the terminal:

```
mkdir -p workdata/Ames_sub1 && cp -r refdata/Ames_sub1/* workdata/Ames_sub1
```

The screenshots for these steps are in the `doc/` folder of the SWAT+ source repository.

Cost

GitHub gives every account 120 free CPU hours per month across all Codespaces instances. A 2-CPU instance running for 60 hours uses 120 hours of quota. Beyond that, you pay per active hour. A small per-hour storage fee applies whether the instance is running or stopped.

Instance states:

- Active: the instance is in use (interactive or compiling). After 30 minutes of inactivity it goes to Inactive. The timeout is configurable.
- Inactive: not running. Storage fee only.
- Shutdown: you stopped it manually. Same as Inactive billing-wise.
- Deleted: no further charges. A new instance has to be created from scratch to come back.

Starting an instance

1. Sign in to GitHub and go to your `swatplus` fork.
2. Open the hamburger menu (top left) and click **Codespaces**.
3. Click **New codespace** (top right).
4. Pick the repository, the branch, and the CPU count. Two CPUs is enough for SWAT+. Click **Create codespace**.

The instance takes a couple of minutes to come up. You should see an install progress bar in the lower right. When it disappears, a drop-down at the top of the editor asks which Fortran compiler to use. Pick one.

Building

Click the CMake icon in the left sidebar (the triangle-with-wrench) to open the CMake pane.

1. Under **Project Status > Configure**, check the Fortran compiler shown. To change it, hover and click the pencil icon, then pick another.
2. Right-click `CMakeLists.txt` in the CMake pane and select **Clean and Reconfigure**. Do this every time you switch compilers, and any time you add or remove a Fortran source file.
3. Right-click `CMakeLists.txt` again and select **Build All Projects**. The Output panel at the bottom should end with:

   ```
   [build] Build finished with exit code 0
   ```

Running and debugging

Click the debug icon in the left sidebar (the triangle with a bug). Pick `Ames_sub1` from the drop-down and click the green triangle.

After a fresh build, VS Code may ask which executable to run. Any of the debug-build options works. A release build is not debuggable.

Watch model output in the **Terminal** tab at the bottom. Set breakpoints or conditional breakpoints, or let it run. On a runtime exception, the debugger stops at the failure point and you can inspect variable values.

Using your own dataset

The `/workdata/my_data` folder starts with a copy of `Ames_sub1`. To use your own inputs, delete its contents and upload yours:

1. In the VS Code Explorer, right-click `/workdata/my_data` and select **Upload**.
2. Allow the cookie prompt if your browser shows one (GitHub needs it to open your local file picker).
3. Select all the files in your input dataset and upload.
4. Switch the debug drop-down to `my_data` and click the green triangle.

To add a third dataset, create a new folder under `/workdata/`, then edit `.vscode/launch.json`. The default file already has blocks for `Ames_sub1` and `my_data`. Copy one block, change `name` and `cwd`, and add a comma between blocks.

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Ames_sub1",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/Ames_sub1",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
```

```
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        },
        {
            "name": "walnut_creek",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/walnut_creek",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

Stopping the instance

To stop billing for active usage, click **Codespaces** in the bottom-left status bar and select **Stop Current Codespace**. It takes a minute or two.

Deleting the instance

After stopping, go back to the browser tab where you created the instance (or open the Codespaces panel from your GitHub repository). Find the instance at the bottom, click the ellipsis on its row, and select **Delete**. Storage charges stop after deletion. A new instance has to be created from scratch the next time you want to work.

Sources

- `.devcontainer/devcontainer.json` and `.devcontainer/Dockerfile` at 62.0.1. `devcontainer.json` contains two `postCreateCommand` keys; only the last one (the `my_data` copy) is used.
- `doc/VSCode_Codespace.md` in the source repository, with the screenshots.

SWAT+ has a scenario regression test: it runs the model on a reference dataset and compares chosen output files with copies kept in the repository. Differences outside a tolerance point to a change in results, which is then either a bug or an intended change that needs new reference outputs.

Test script

`test/spcheck.py` (Python 3, standard library only) has three commands:

Command	Use
`ctest <executable> <reference folder> <work folder>`	copy the reference folder into the work folder, run the executable there, and compare the files listed in the folder's `.testfiles.txt` with the reference copies; exit with code 1 if any value differs
`run <executable> <scenario>`	run `build/<executable>` on `data/<scenario>` in a new numbered folder under `runs/<scenario>/`
`compare <scenario> <a> <b>`	compare two numbered runs of a scenario

Options `--abserr` and `--relerr` set the tolerances (defaults `1e-8` and `0.05`), and `--nlines` and `--nerrorlines` limit how much of each file is checked and reported.

Two numbers are equal when

$$|v_1 - v_2| \leq e_{abs} + e_{rel}\,|v_2|$$

Only real numbers are compared, value by value on matching lines; integers and text are skipped, as are the title lines with the build date (lines containing `MODULAR`).

Running the test at 62.0.1

`CMakeLists.txt` registers one test, `Ames_sub1`, with tolerances of `1e-8` absolute and 1 % relative, and reference data in `data/Ames_sub1`. At 62.0.1 the repository has no `data/` folder; the reference datasets are in `refdata/`. `ctest` therefore fails at once with `data dir not found`.

To run the comparison, call the script directly with the `refdata` folder:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
cmake --build build -j 8
python3 test/spcheck.py ctest build/swatplus-*-Rel refdata/Ames_sub1 build/test-runs --abserr
 ↪  1e-8 --relerr 0.01
```

Notes on the reference dataset:

- A full `Ames_sub1` run writes about 9 GB, mostly daily carbon output, and takes a few minutes. Make sure the work folder has the space.
- `.testfiles.txt` lists 42 files. With the shipped `print.prt` the model rewrites fewer than half of them; the rest (all `basin_*` files, `hydin_aa.txt`, `ru_aa.txt` and others) are copied from the reference folder and compared with themselves. The test therefore checks the HRU water and nutrient balances, plant and yield output and management output, and not channel, aquifer or basin output.

Adding a scenario

1. Put a complete input set in a new folder under `refdata/` (or `data/`).
2. Run the model and check the outputs you want to fix as the reference.
3. Commit the inputs and those outputs.
4. Add a `.testfiles.txt` naming the output files to compare, one per line (`#` starts a comment).
5. Add an `add_test` line to `CMakeLists.txt` pointing at the folder.

Continuous integration

`.github/workflows/test.yml` builds SWAT+ with Intel `ifx` on Ubuntu and runs `ctest` on pushes to branches whose name ends in `main`. With the `data/` path above, the test step fails. The release workflow (`build.yml`) builds the release executables for each tag and does not run the tests.

Sources

- test/spcheck.py: `ctest_all`, `cmp_file` and the `run`/`compare` commands.
- CMakeLists.txt: `set(ref_dir "${PROJECT_SOURCE_DIR}/data")`, tolerances `aerr = 1e-8`, `rerr = 0.01`, and `add_test(Ames_sub1 ...)`. The `Ithaca_sub6` test is commented out.
- refdata/Ames_sub1/.testfiles.txt.
- .github/workflows/test.yml and .github/workflows/build.yml.
- The 9 GB output size and the list of files rewritten are from a run of the 62.0.1 `gnu-lin_x86_64` release executable on 20 October 2026.

A SWAT+ version is a Git tag. No source file is edited to make a release: the build reads the tag and writes it into the executable's name and its start-up banner.

Version numbers

Current tags have three parts, revision, minor and patch, for example `62.0.0` and `62.0.1`. Older tags include `61.0`, `61.0.2.61` and `60.5.7`. List them with:

```
git tag --sort=-creatordate | head
```

At configure time CMake runs `git describe --tags`:

- on a tagged commit the version is the tag, for example `62.0.1`;
- on a later commit it is the tag, the number of commits since, and the abbreviated commit name, for example `62.0.1-3-g1a2b3c4`;
- outside a Git clone it is `unknown`.

A version can also be given by hand with `-D TAG=<version>` when configuring.

Version in the executable and outputs

The executable name, `swatplus-<version>-<compiler>-<system>_<arch>[-Rel|-Dbg]`, for example `swatplus-62.0.1-gnu-l`

The start-up banner, printed to the screen and to `simulation.out`:

```
                  SWAT+
             Revision 62.0.1
      Soil & Water Assessment Tool
GNU (13.3.0), 2026-10-19 22:51:37, Linux
    Program reading . . . executing
```

The title line of every output file, which contains the build date and `Rev <year>.<version>`:

```
 demo                        SWAT+ 2026-10-19        MODULAR Rev 2026.62.0.1
```

Building a tagged version

```
git checkout 62.0.1
cmake -B build -D CMAKE_BUILD_TYPE=Release
cmake --build build -j 8
```

A tag on GitHub can be moved after it is first published (the 62.0.1 tag was replaced on 19 October 2026). Run `git fetch --tags --force` before checking out a tag in an existing clone, so that the local tag matches the one on GitHub.

Making a release

1. Merge the changes for the release into `main`.
2. Tag the merge commit and push the tag:

   ```
   git tag 62.0.2
   git push origin 62.0.2
   ```

3. Pushing a tag of the form `x.y` or `x.y.z` starts the release workflow, which builds the executables for Windows, Linux and macOS with gfortran and the Intel compilers and attaches the zipped executables to a GitHub release.
4. Write the release notes on the GitHub release page, and archive the release on Zenodo (see Citing and license).
5. Tag the documentation repository with the same version, which publishes a matching version of this site.

Sources

- CMakeLists.txt: `if(NOT TAG)` block with `git describe --tags`, `SWATPLUS_EXE`, and `configure_file` of `src/main.f90.in` with `ISO`, `TODAY` and `YEAR` time stamps.
- src/main.f90.in: `prog = " SWAT+ @TODAY@ MODULAR Rev @YEAR@.@SWAT_VERSION@"` and the banner format. The generated `src/main.f90` is overwritten on every configure; edit `main.f90.in`.
- doc/Tagging.md.
- .github/workflows/build.yml: runs on tags matching `*.*` and `*.*.*`.
- Banner and title line: output of the 62.0.1 `gnu-lin_x86_64` release executable.

Conventions for writing SWAT+ Fortran source. The goal is portable, readable and reliable code that compiles cleanly on every supported compiler and operating system.

Each item is tagged with one of three levels:

- Required: mandatory. Pull requests that violate these will not be merged.
- Recommended: strongly preferred. Deviations need justification.
- Encouraged: optional. Apply for consistency where reasonable.

The list below is ordered by decreasing impact on code quality. Most subjective style items (indent width, capitalization) are in the lower tiers.

Interoperability and portability

Required

- Source must conform to the ISO Fortran 95 standard.
- No compiler-specific or platform-specific extensions.
- No use of compiler-dependent error specifier values (`IOSTAT`, `STAT`).
- The source must compile with `gfortran` from GCC, even if the production build uses the Intel compiler.
- `EXIT(N)` is the F90/95 standard exit. Prefer it to `STOP`. `STOP` does not always return an error code. If you need an error code passed back to a script, use `EXIT`, and keep its call sites in one central place.

Recommended

- For floating-point precision and integer size, use the F90/95 `KIND` feature. Do not depend on a vendor-supplied default precision flag.
- Do not use tab characters. Tabs are not part of the Fortran character set and break portability.

Encouraged

- For code that interacts with external frameworks, declare every variable with a `KIND` type to make integration easier.

Readability

Required

- Free-form source.
- Consistent indentation, at least two spaces per level.
- Organize source into modules.
- Do not use Fortran keywords (`DATA`, etc.) as variable names.
- Use meaningful names for variables and parameters. Recognized abbreviations are fine when full names would be unwieldy.
- Every externally-called function or subroutine has a header describing what it does, its arguments, and the author. Small internal routines can use a few descriptive comments instead.
- No magic numbers. Physical constants (pi, gas constants) go in `PARAMETER` statements, not hard-coded into expressions.
- Do not hard-code numeric literals in argument lists. The compiler's default precision for literals is not guaranteed.
- No `GOTO`. If you must use one, document why.

Recommended

- Name your loops and other constructs (subroutines, functions, modules, interfaces) where the name aids readability, especially in nested loops.
- Comment input, output, and local variables. Grouping comments across similar variables is fine when their names speak for themselves.
- Comment to mark major functional sections.
- Do not use Fortran statement names or intrinsic function names as symbols.
- Prefer named parameters over numeric literals: `REAL, PARAMETER :: PI = 3.14159, ONE = 1.0`.

Encouraged

- In new code, follow a consistent style. When editing existing code, match the style around you.
- Indent comments the same way as code.
- Group related procedures and data into modules.
- Keep lines to 132 characters or fewer.
- Use `<, >, <=, >=, ==, /=` instead of `.lt., .gt., .le., .ge., .eq., .ne.`.
- Name modules the same as the file that contains them. Avoid putting multiple modules in one file.
- Use blanks around operators, around `=`, and in declarations to separate syntax elements.
- Always use the `::` notation in declarations, even when there are no attributes.
- Line up attributes, variable names, and trailing comments vertically in declarations.
- Remove unused variables.
- Remove debug code once the bug is fixed.

Reliability

Required

- `IMPLICIT NONE` in every program, module, and procedure.
- Use `PRIVATE` in modules by default, then explicitly mark what should be `PUBLIC`. Exception: modules that exist only to expose public data (constants, for example) can stay fully public.
- Initialize every variable. Do not rely on the machine's default.
- Do not initialize a variable of one type with a value of another type.

Recommended

- Do not use `==` or `/=` on floating-point values. Compare against an explicit tolerance instead (an epsilon check).
- In mixed-type expressions and assignments, write the type conversion explicitly. Do not let the compiler do it implicitly.

- No `INCLUDE` files. Use `USE` with modules.
- Derived types belong in a separate module, together with the procedures that operate on them.
- Each procedure should do one thing.
- Module-level (global) public variables should contain only static or rarely-changing data.

Encouraged

- Always use parentheses to make expression evaluation order explicit.
- Prefer derived types (`%` field access) over loose collections of arrays.

Arrays

Required

- Subscript expressions must be integer.
- Do not assume any particular array passing mechanism when passing arrays as arguments.

Recommended

- Use array operations and intrinsics where they apply.
- Pass arrays as assumed shape.

Encouraged

- Declare `DIMENSION` on every non-scalar.

Dynamic memory and pointers

Required

- Prefer `ALLOCATABLE` arrays over pointers where possible. Reduces leak and fragmentation risk.
- Pointers are allowed when a subroutine needs to return a declared array to its caller.
- Initialize every pointer to `NULL()` in its declaration: `INTEGER, POINTER :: x => NULL()`.
- Prefer automatic arrays for transient allocations. `ALLOCATABLE` and `POINTER` arrays need explicit `DEALLOCATE`.

Recommended

- Always deallocate. Especially inside subroutines and loops.
- Check the `STAT` argument on `ALLOCATE` and `DEALLOCATE`.
- Do not repeatedly `ALLOCATE`, `DEALLOCATE`, then `ALLOCATE` a larger block. Heap fragmentation follows.

Encouraged

- Dynamic allocation is preferred over fixed-size declarations with worst-case dimensions.
- Use automatic arrays in subroutines for simplicity.

Looping

Required

- No `GOTO` for loop exit or continuation. Use `EXIT` or `CYCLE`.

Recommended

- No numbered `DO` loops (`DO 10 ... 10 CONTINUE`).

Functions and procedures

Required

- Avoid `SAVE`. Keep state in module variables instead.
- No `ENTRY` statements in functions.
- Functions must not return pointers.
- Do not name a user function the same as an intrinsic (`SUM`, etc.).
- A procedure that returns a single value should be a function. The value can be a derived type.
- Communicate through arguments or through the variables of the procedure's module.

Recommended

- Every dummy argument that is not a pointer has an `INTENT`.
- Avoid type-specific intrinsics (`AMAX`, `DMAX`). Use the generic (`MAX`).
- Do not declare static-dimensioned array arguments.
- Validate argument values where it matters.

Encouraged

- On an error, print a message that includes the routine name. You may terminate inside the routine, or pass an error flag back through the argument list.
- Use overloading when the same operation works on multiple types.
- Use modules or `CONTAINS` for explicit interfaces.
- Avoid external routines. They need separate interface blocks that drift out of sync.

I/O

Required

- External-file I/O statements include `err=`, `end=`, and `iostat=` as appropriate.
- All global variables, if any, are initialized at startup.

Recommended

- Avoid `NAMELIST` I/O.
- Use `WRITE` instead of `PRINT` for non-terminal I/O.
- Use character parameters or in-line format specifiers in `READ` and `WRITE`. Do not use labeled `FORMAT` statements.

Obsolescent features to avoid

Required

- No common blocks. Use modules.
- No assigned or computed `GO TO`. Use `CASE`.
- No arithmetic `IF`. Use a block `IF`.
- Use `REAL`, not `DOUBLE PRECISION`.
- Avoid `DATA`, `ASSIGN`, labeled `DO`, `BACKSPACE`, blank `COMMON`, `BLOCK DATA`.
- Do not branch to an `END IF` from outside its `IF` block.
- No non-integer `DO` loop counters.
- No Hollerith constants.
- No `PAUSE`.
- No multiple `RETURN`, no alternate `RETURN`.

Recommended

- No `EQUIVALENCE`. Use pointers or derived types.

Encouraged

- Do not implicitly reshape arrays when passing them. This was common in F77 and is forbidden by F90 but still possible through external routines without interface blocks.

Source files

Required

- Document every function interface: argument name, type, units, description, constraints, defaults.
- No `INCLUDE`. Use `USE`.
- Keep lines (including comments) under 80 columns where the language allows it.

Recommended

- Keep individual procedures under 300-500 effective lines.
- Separate statement blocks with blank lines or a marker character in column 1.
- Indent consistently.
- Module and subprogram names are lower case. The file is named after the contained module or subprogram, with `.f90`.

Encouraged

- Separate argument declarations from local declarations visually.
- Use descriptive unique names. Keep them under 12-15 characters.
- Indent continuation lines so a multi-line expression lines up readably.
- Comments start with a marker character (`!`). A standalone comment line starts the marker in column 1.

References

- SAS Coding Conventions
- WRF Coding Conventions
- USGS Coding Standards

  Sources

  - `doc/coding_conventions.md` in the source repository.

Code Of Conduct

The SWAT+ project follows the Contributor Covenant. Everyone taking part in the project, in issues, pull requests, reviews and discussions, is expected to follow it.

The full text, including how to report a violation, is in `CODE_OF_CONDUCT.md` in the source repository.

Sources

- `CODE_OF_CONDUCT.md` and `doc/code_of_conduct.md` at 62.0.1.

Repositories

- Model source: github.com/swat-model/swatplus, with issues, pull requests and releases.
- This documentation: github.com/swat-model/swatplus-documentation.

Questions about using the model are best asked through the SWAT support page.

Reporting a problem

Open an issue with:

- the SWAT+ version (shown in the banner at start-up and in the executable name);
- the operating system and, for your own builds, the compiler;
- a project that shows the problem, as small as possible (one of the `refdata` datasets with the changes that cause it is ideal);
- `simulation.out` and `diagnostics.out` from the run.

Contributing code

1. Fork `swat-model/swatplus` and create a branch from `main` with a descriptive name (`fix-aquifer-init`, `add-soil-temp-output`).
2. Build and run your change on the reference datasets before opening a pull request. See Build with CMake and Testing.
3. Open a pull request against `main`. Describe what the change does and why, which datasets you ran and how their outputs changed, and any change to input or output files.
4. Answer review comments on the same branch.

Reviewers look for:

- a clean build with gfortran and the Intel compilers;
- unchanged outputs for parts of the model the change does not touch;
- defaults for new parameters that keep existing results when the parameter is not set;
- input file changes matched by a SWAT+ Editor change or a note on how to update existing projects;
- code that follows the coding conventions.

Contributing to the documentation

The documentation is a MkDocs site in the `swatplus-documentation` repository. Each page states the SWAT+ commit it was checked against in its front matter (`verified_at_commit`), and every statement about model behaviour should come from the source at that commit. Page bodies are written for model users; source file names and variable names go in the collapsed Sources block at the end of each page (`??? quote "Sources"`).

To check a change locally:

```
pip install -r requirements.txt
mkdocs serve
```

The site is versioned with mike. Pushing a tag to the documentation repository, named after the SWAT+ version the pages describe (for example `62.0.1`), publishes that version and moves the `latest` alias to it. Earlier versions stay available from the version selector.

Sources

- CI: `.github/workflows/test.yml` and `.github/workflows/build.yml` in the source repository.
- Documentation build and deployment: `mkdocs.yml`, `requirements.txt` and `.github/workflows/deploy.yml` in the documentation repository.